ESCUELA POLITÉCNICA NACIONAL

INGENIERÍA DE SOFTWARE

Aplicaciones Móviles

ISWD713

GR3SW

Sara Guayasamin

Sebastián Ramos

Mateo Quisilema

2026-A

FECHA

15 de abril de 2026

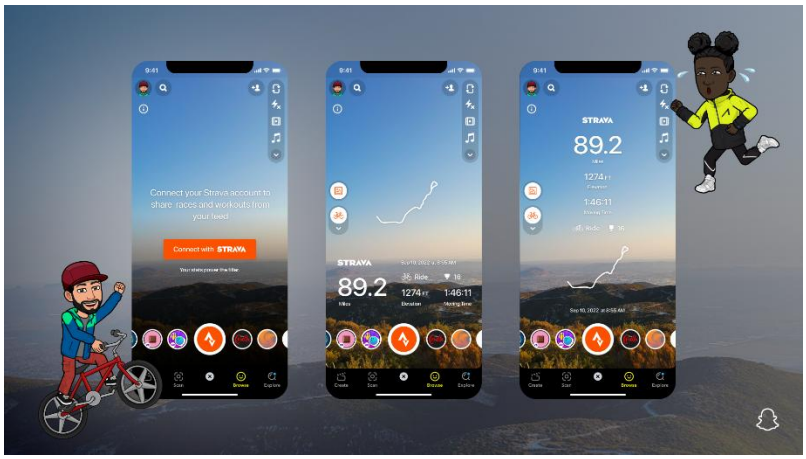
Aplicaciones elegidas:

- Strava
- BeReal

Parte 1: Relación con la Evolución del Desarrollo Móvil

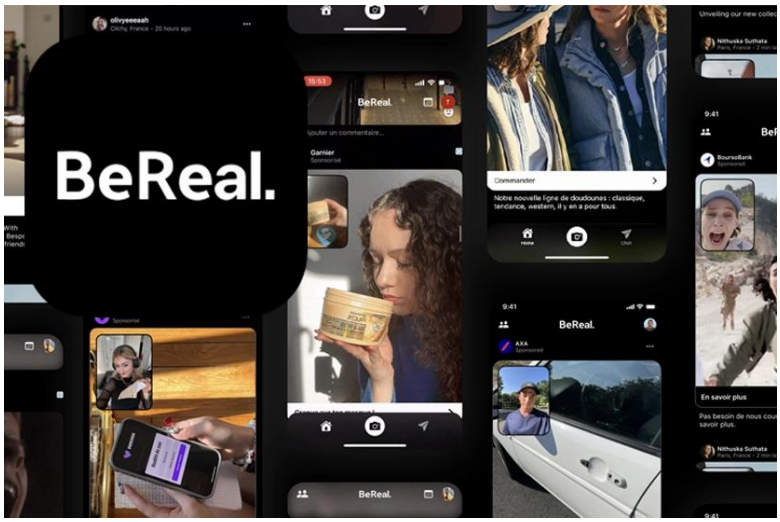
El desarrollo de aplicaciones móviles ha atravesado distintas etapas arquitectónicas que responden a las necesidades de cada momento. Al hacer el análisis de las mismas podemos entender como estas decisiones siguen teniendo relevancia actualmente o como han cambiado con el tiempo.

En el caso de Strava, se observa un claro enfoque hacia el desarrollo nativo puro. Esta decisión está directamente relacionada con la naturaleza de la aplicación, la cual requiere acceso constante y preciso a recursos del dispositivo como el GPS, sensores de movimiento y procesamiento en segundo plano. El uso de lenguajes como Kotlin y Swift permite una integración profunda con el SO (sistema operativo), optimizando tanto el rendimiento como el consumo energético. Esto demuestra que, a pesar del avance de tecnologías multiplataforma, el desarrollo nativo sigue fundamental en aplicaciones que dependen intensamente del hardware.



Por otro lado, BeReal también se apoya en un enfoque nativo, pero con menores exigencias técnicas en comparación de Straba. La aplicación requiere acceso inmediato a

la cámara frontal y trasera de forma simultánea, así como una sincronización precisa con notificaciones en tiempo real. Características que hacen que el desarrollo nativo sea la opción más adecuada, porque da una respuesta más rápida y control directo en los recursos. Eso sí, es importante mencionar que a diferencia de Strava, es posible que BeReal use componentes híbridos o WebView en secciones menos críticas, como configuraciones o interfaces secundarias.



Respecto al uso de puentes entre código nativo y JS, Strava y BeReal parecen evitar este tipo de arquitectura en sus componentes principales. Esto porque el uso de un puente introduce latencia en la comunicación entre capas, lo cual puede afectar negativamente la experiencia del usuario en aplicaciones que requieren procesamiento en tiempo real o respuestas inmediatas. En Strava por ejemplo, esto podría afectar la precisión del tracking, mientras que en BeReal podría romper la sincronización exacta del momento de captura.

Finalmente, al considerar el futuro del desarrollo móvil, incluyendo dispositivos plegables, realidad aumentada y weareables, se evidencian diferencias importantes entre amabas aplicaciones. Strava tiene un mayor potencial a tecnologías weareables y realidad aumentada debido a su enfoque en datos de actividad física en tiempo real. Sin embargo, esto también implica la necesidad de arquitecturas reactivas que puedan adaptarse

dinámicamente a cambios de la interfaz. BeReal, en cambio, presenta una estructura más simple, lo que facilita su adaptación a pantallas plegables, aunque su potencial en entornos de realidad aumentada es más limitado.

Parte 2: Taller Práctico - Análisis Crítico de Aplicaciones

1. **Ingeniería Inversa de la Decisión:**

El análisis de aplicaciones líderes revela una fuerte preferencia por el desarrollo nativo cuando el hardware y el rendimiento son críticos.

- **El Caso Strava:** Su arquitectura exige precisión ininterrumpida del GPS, procesamiento continuo de datos en segundo plano (*background processing*) y un estricto control sobre el consumo energético. La adopción de un framework híbrido en este escenario introduciría latencia y limitaría el acceso de bajo nivel al hardware, comprometiendo la fiabilidad de la telemetría. Por tanto, el desarrollo nativo es la única vía para garantizar rendimiento, precisión y eficiencia energética.
- **El Caso BeReal:** El núcleo de su propuesta de valor es capturar el "momento exacto". Esto requiere un acceso casi instantáneo a los controladores de la cámara y una sincronización perfecta con los servicios de notificaciones *push*. Cualquier retraso derivado de un puente de comunicación (*bridge*) en un framework híbrido rompería la experiencia de usuario. El enfoque nativo le otorga la respuesta inmediata necesaria.

2. **Análisis de Rendimiento y Arquitectura:**

Ambas plataformas enfrentan desafíos únicos que requieren optimizaciones específicas.

- **Strava:** El problema clave radica en mantener una alta precisión de rastreo sin agotar los recursos del dispositivo. Los cuellos de botella se encuentran en la lectura continua del GPS y el procesamiento de rutas, velocidad y distancia. La solución técnica pasa por una optimización nativa profunda y una gestión eficiente del ciclo de vida de los sensores.
- **BeReal:** Su mayor reto es la concurrencia masiva. Al enviar una notificación global, la aplicación debe gestionar el acceso simultáneo a la cámara dual en millones de dispositivos y la posterior subida de imágenes. La latencia de red y la congestión de los servidores en horas pico son sus principales cuellos de botella. Esto se mitiga mediante un *backend* altamente escalable y el uso de APIs nativas que optimizan el tiempo de respuesta del hardware.

3. **El Desafío de las Nuevas Pantallas:**

Frente a las nuevas tecnologías de visualización, la capacidad diverge:

- **Strava:** Se encuentra en una posición ventajosa para integrar Realidad Aumentada (AR) compleja, proyectando datos en tiempo real sobre el entorno. Sin embargo, adaptar su densa interfaz de usuario a pantallas plegables (*foldables*) representa un reto de diseño mayor.
- **BeReal:** Su interfaz sencilla facilita la transición a dispositivos *foldables*, pero tiene un potencial limitado para experiencias complejas de AR.

Para implementaciones avanzadas de AR tecnologías como React Native o Flutter resultan insuficientes; se requiere desarrollo nativo puro o la integración de motores gráficos dedicados como Unity.

4. **Veredicto Crítico:**

Para el desarrollo de una aplicación que demande renderizado de mapas 3D, gestión intensiva de notificaciones concurrentes y soporte fluido para dispositivos plegables, el **desarrollo nativo** es la elección indiscutible. Garantiza acceso directo a las APIs del sistema, un rendimiento superior en renderizado gráfico, menor latencia y una integración robusta con los sensores del dispositivo, superando las limitaciones inherentes a frameworks como React Native o Flutter en escenarios de alto estrés computacional.

5. **¿Qué es un aplicativo nativo, hay varios, como se clasifican?**

Una aplicación nativa es aquella desarrollada específicamente para un sistema operativo (OS) utilizando los lenguajes, SDKs y herramientas oficiales provistas por el creador del

OS (ej. Apple o Google). Esto permite una compilación directa al código máquina del dispositivo, asegurando el máximo rendimiento.

El ecosistema de desarrollo móvil se clasifica de la siguiente manera:

1. **Nativo Puro:** Desarrollo con lenguajes oficiales (Kotlin/Java para Android; Swift/Objective-C para iOS).
2. **Multiplataforma Híbrido (WebView):** Aplicaciones web empaquetadas en un contenedor nativo (ej. Cordova, Ionic).
3. **Multiplataforma con Puente (Bridge):** Traducen componentes de UI al vuelo, comunicándose con los hilos nativos mediante un puente (ej. React Native).
4. **Multiplataforma con Motor de Renderizado Propio:** Dibujan sus propios componentes visuales directamente sobre un lienzo en la pantalla, eludiendo la UI nativa (ej. Flutter).